Multi Resolution Fast Feedforward Networks

Filippo di Gravina, 840297, Deep Learning

June 18, 2026

Abstract

A Fast Feedforward layer replaces a dense layer of width 2^D by a depth-D binary tree, so that each input is routed down a single path and inference costs $\mathcal{O}(\log(\text{width}))$ instead of $\mathcal{O}(\text{width})$. The tree chooses which expert evaluates an input, but spends the same depth on every input. This report develops MR-FFF, which makes both the arity and the depth of the tree input dependent. Each node carries a halting score $\lambda(\mathbf{x}) = \sigma(\mathbf{w}^\top \mathbf{x} + b)$ that is a function of the input, so two inputs reaching the same node may stop at different depths. We show that a halting score that does not depend on the input cannot learn to stop, derive a training objective as the expected loss over the induced distribution of stopping nodes, and add an expected depth term that exposes the accuracy against compute tradeoff. On the four tested datasets, MR-FFF remains slower than FFF in real compute time, because at these small widths the per step control overhead dominates, but better performs in terms of accuracy.

1 Fast Feedforward Networks

An FFF layer with input dimension d, output dimension m and depth D is a complete binary tree of $2^D - 1$ linear routers and 2^D linear leaf experts $E_\ell(\mathbf{x}) = W_\ell \mathbf{x} + \mathbf{c}_\ell$ [1]. At training time, the decision at router i is relaxed to a probability $c_i(\mathbf{x}) = \sigma(\mathbf{w}_i^\top \mathbf{x} + b_i)$ of branching right, the probability of reaching a leaf is the product of the branch probabilities along its path, and the layer returns the leaf mixture $\hat{\mathbf{y}}(\mathbf{x}) = \sum_\ell \Pr[\text{reach } \ell \mid \mathbf{x}]\, E_\ell(\mathbf{x})$. This is differentiable but evaluates all 2^D leaves, hence the training pass is $\mathcal{O}(\text{width})$. At test time the sigmoids are thresholded, the input descends one path of length D and a single leaf is evaluated, so a hard forward pass costs

$$\text{FLOP}_{\text{FFF}} = \underbrace{2Dd}_{\text{routers}} + \underbrace{2dm}_{\text{leaf}}. \tag{1}$$

A dense layer of width h costs $2dh + 2hm$. Matching capacity by setting $h = 2^D$, the dense cost grows linearly in h while Eq. 1 grows only logarithmically, which is the source of the asymptotic advantage. The count in Eq. 1 measures arithmetic alone and ignores that the D routing steps are not parallelizable and are control heavy, making them slower in the routing evaluations.

2 MR-FFF

2.1 Structure

MR-FFF generalises the binary tree to a K-ary tree of maximum depth D stored as a heap, where the children of the node at index f are $Kf + 1, \ldots, Kf + K$. Every node, not only the leaves, owns a linear expert, so an input may produce its output and terminate at any node. Each internal node owns a router $r_n(\mathbf{x}) = \text{softmax}(R_n \mathbf{x} + \mathbf{b}_n^r)$ over its K children and a halting score $\lambda_n(\mathbf{x}) = \sigma(\mathbf{w}_n^{h\top} \mathbf{x} + b_n^h)$. The halting score is a function of the input rather than a parameter of the node alone, which is what allows the depth to vary between two inputs that happen to reach the same node.

2.2 Node constant halting scores

Suppose the halting score at node n were a single learnable scalar g_n shared by every input. Conditioned on reaching n, the soft output is $g_n E_n(\mathbf{x}) + (1 - g_n)\, S_n(\mathbf{x})$, where S_n is the mixture produced by the subtree rooted at n. The subtree can place all of its mass on one child whose expert copies E_n, so the function class of S_n contains E_n and strictly extends it. The smallest reconstruction loss attainable with the subtree is therefore no larger than the one attainable with E_n, and the loss is non increasing as mass shifts away from n. Consequently, the unpenalized objective is minimized at $g_n \to 0$, meaning the model never halts, and only a large halting penalty can raise g_n. Because raising g_n forces the use of the weaker single expert in place of the richer subtree, that penalty buys early stopping only at the cost of accuracy. Making λ_n depend on $\mathbf{x}$ makes the model halt on the inputs for which E_n already matches the target while continuing on the rest.

2.3 Objective

Unrolling the tree level by level, let ρ_n be the probability that the input reaches node n, with $\rho_{\text{root}} = 1$. The mass at n splits into a halting part and a continuing part that the router distributes to the children,

$$p_n = \rho_n\, \lambda_n(\mathbf{x}), \qquad \rho_c = \rho_n \big(1 - \lambda_n(\mathbf{x})\big)\, [r_n(\mathbf{x})]_{k(c)}. \tag{2}$$

Setting $\lambda \equiv 1$ at the deepest level forces all remaining mass to halt, which makes the halting masses a probability distribution over the terminating node, $\sum_n p_n = 1$. The training loss is the expected cross-entropy under that distribution,

$$\mathcal{L}_{\text{rec}} = \sum_n p_n\, \text{CE}(E_n(\mathbf{x}),\, y)\,. \tag{3}$$

Each expert receives a gradient proportional to its own halting mass p_n, so every node is trained as a predictor. To express a preference for shallow computation, the expected halting depth is added as a cost, and to control the effective arity we penalized the router entropy weighted by the mass that flows through each node,

$$\mathcal{L}_{\text{ponder}} = \sum_n p_n\, \text{depth}(n), \qquad \mathcal{L}_{\text{arity}} = \frac{\sum_n \rho_n \big(1 - \lambda_n\big)\, \mathcal{H}(r_n(\mathbf{x}))}{\sum_n \rho_n \big(1 - \lambda_n\big)}. \tag{4}$$

The expected depth is a differentiable proxy for inference cost, and a router with low entropy concentrates on few children and so has a low effective arity. The full objective is:

$$\mathcal{L} = \mathcal{L}_{\text{rec}} + \lambda\, \mathcal{L}_{\text{ponder}} + \mu\, \mathcal{L}_{\text{arity}}, \tag{5}$$

and the single coefficient λ, called reg_split in the code, moves the model along the accuracy against depth trade-off.

2.4 Inference

At inference time, the input follows a single tree path. It halts at the first node whose score exceeds one half and otherwise descends to the highest scoring child, with a forced halt at depth D, as stated in Algorithm 1. Inputs therefore evaluate a variable number of nodes, and the expected cost is $\bar{L}\,(2Kd + 2d) + 2dm$, where $\bar{L}$ is the average path length.

Algorithm 1 MR-FFF hard inference

$n \leftarrow$ root, $\quad L \leftarrow 0$
for 1 **to** D **do**
  if $\sigma(\mathbf{w}_n^{h\top}\mathbf{x} + b_n^h) > \frac{1}{2}$ **then break**
  end if
  $k \leftarrow \arg\max_j [R_n\mathbf{x} + \mathbf{b}_n^r]_j, \quad n \leftarrow Kn + 1 + k, \quad L \leftarrow L + 1$
end for
return $E_n(\mathbf{x})$ and path length L

3 Experimental setup

We use four OpenML classification datasets with stratified split and features standardised on the training split. Spambase has 57 features and 2 classes, USPS has 256 features and 10 classes, letter has 16 features and 26 classes, and satimage has 36 features and 6 classes. All models train with Adam at learning rate 10^{-2} and weight decay 10^{-4}, reduced to 10^{-3} for MR-FFF, with batch size 256 for 20 epochs and a single seed. The baselines are a depth-10 FFF with 1024 leaves and two dense networks of width 1024, one with a single hidden layer denoted MLP1, and one with two hidden layers denoted MLP2.

4 Results

4.1 FFF and MR-FFF Inference Time

To benchmark the computational efficiency of FFF against a standard baseline, we compare an FFF and a same width dense MLP across widths from 24 to 212 (d=64, 10 classes, batch size 1024 Figure 1). While the analytical MLP to FFF FLOP ratio scales from 1.3 at width 16 to 215 at width 4096 (Eq. 1), empirically the compute time does not perfectly follow a logarithmic scale.

Below a crossover threshold near width 1024, the FFF is slower than the dense layer despite its lower FLOP count. The FFF yields actual speedups only at widths 2048 (2.5×) and 4096 (4.9×). This latency is due to the sequential routing steps, because they introduce a fixed kernel launch and memory overhead. A single large matrix multiplication avoids this sequential bottleneck, remaining more efficient until the matrix size becomes large enough for arithmetic FLOPs to dominate execution time.

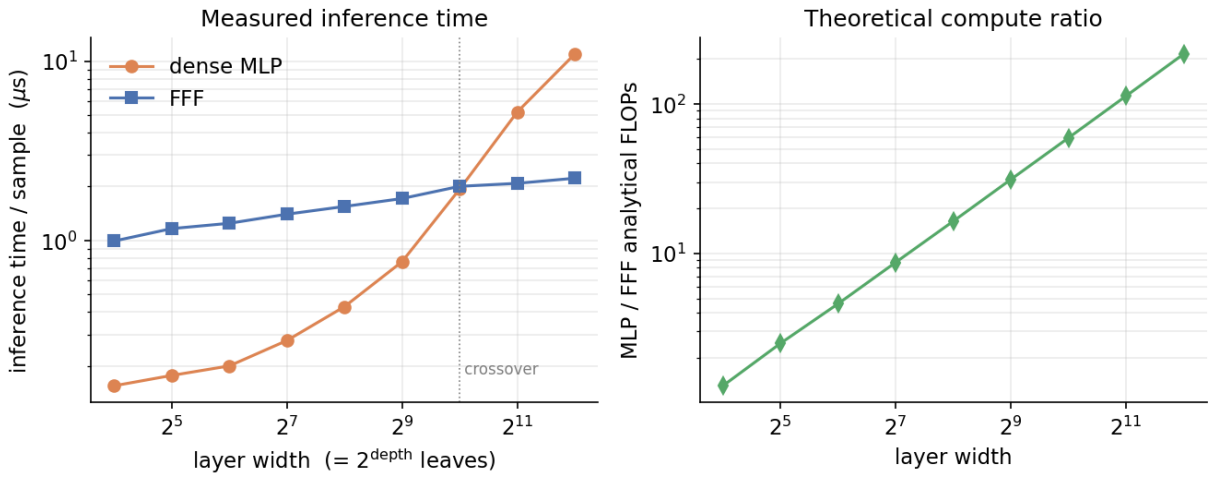

Figure 1: Left: Measured per sample execution time (logarithmic axes). Right: Analytical MLP to FFF FLOP ratio.

4.2 Fixed Arity

Disabling the halting mechanism freezes the tree structure into a standard, fixed-depth K-ary FFF, isolating the effect of branching arity. We compare three configurations maintaining 64 leaves (K,D) combinations of (2,6), (4,3), and (8,2) against a depth-6 binary FFF baseline. Figure 2 and Table 1 summarize the performance.

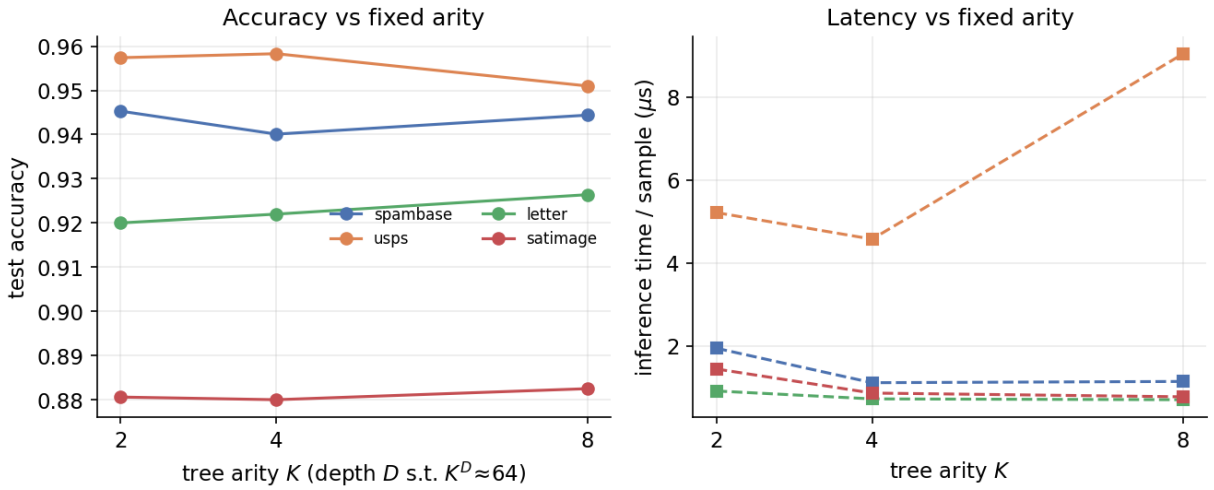

Figure 2

Increasing arity consistently improves accuracy over the binary baseline. On the letter dataset, accuracy rises from 0.825 to approximately 0.92. Wider and shallower trees match the representation capacity of deeper structures while mitigating sequential routing overhead. For example, on the satimage dataset, the (8,2) configuration achieves 0.8825 accuracy in 0.77 μs, whereas the (2,6) configuration requires 1.44 μs to reach 0.8806. At a fixed leaf budget, trading tree depth for higher arity reduces latency with no degradation in task performance.

4.3 Learned depth

Enabling input-dependent halting with $K = 4$ and $D = 3$ and sweeping the depth coefficient λ over 0, 0.02 and 0.1 shows that the MR-FFF correctly learns its structure using the data. At $\lambda = 0$ the fraction of inputs reaching maximum depth is between 0.36 and 0.69. Increasing λ shortens the paths monotonically and drives $f_{\max}$ toward zero, as Figure 3 shows. The reduction is nearly free over a useful range. On spambase, the accuracy stays at 0.9427 while the average path falls from 2.12 to 1.03 as λ goes from 0 to 0.02, and on USPS the operating point at $\lambda = 0.02$ is both shorter and more accurate than the one at $\lambda = 0$, reaching 0.9557 at $\bar{L} = 0.82$ against 0.9544 at 1.69, so a small amount of halting pressure also acts as a regularizer. The accuracy against-depth graph in Figure 4 shows the learned models sitting above and to the left of the depth-6 FFF on every dataset, reaching equal or higher accuracy at one to two levels of depth instead of six.

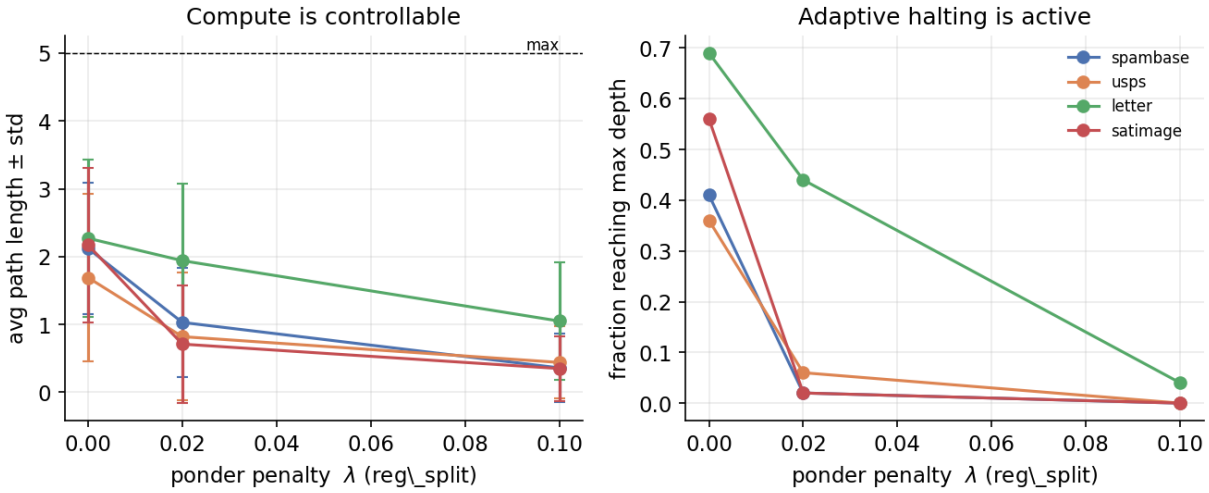

Figure 3: Left: average path length as λ grows. Right: fraction of inputs reaching maximum depth.

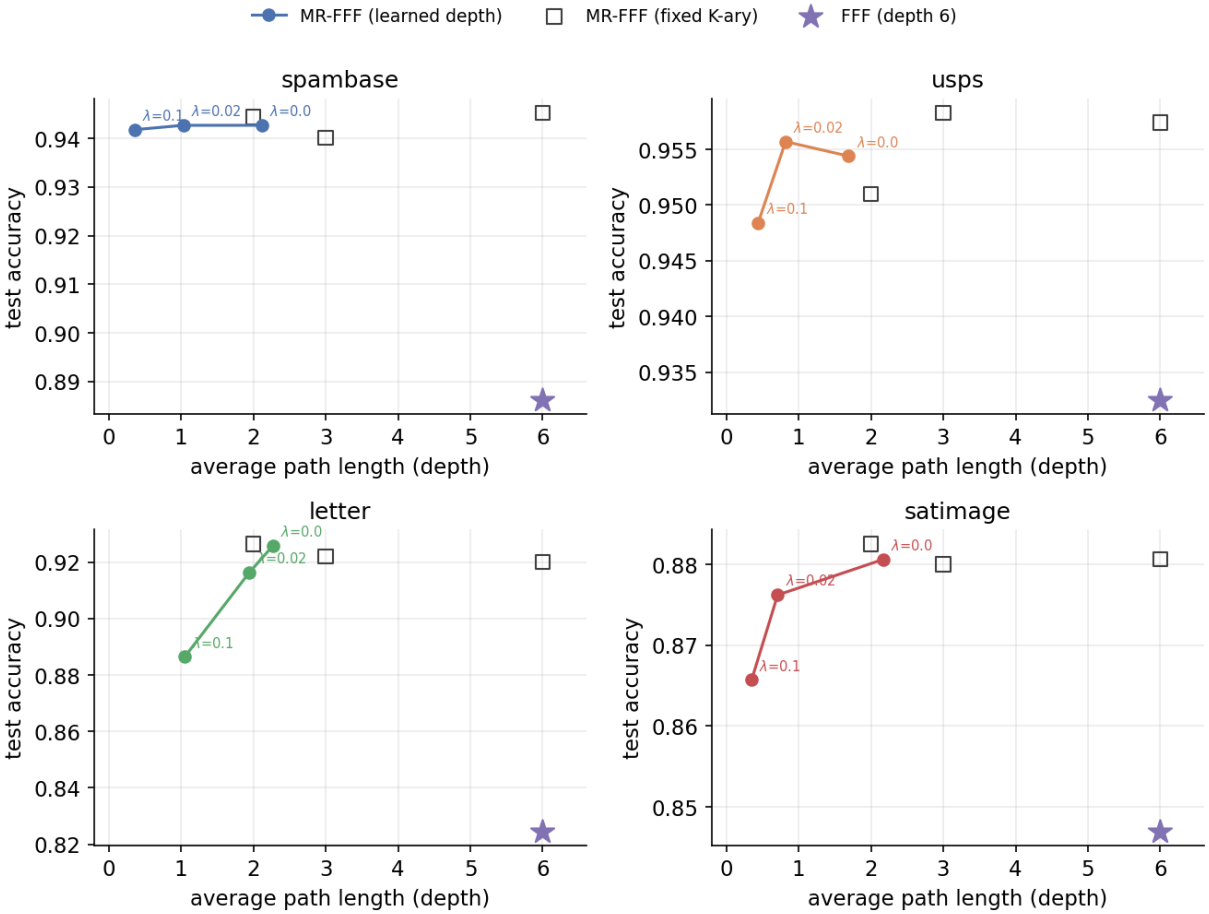

Figure 4: Accuracy against average path length. Filled circles are learned MR-FFF at the three values of λ, open squares are fixed K-ary trees, and the star is the depth-6 FFF reference.

4.4 Benchmark

Table 2 and Figure 5 compare a depth-5, arity-4 MR-FFF against the depth-10 FFF and the two dense networks. MR-FFF improves on FFF on three of the four datasets, most clearly on satimage at 0.840 against 0.777 and on USPS at 0.963 against 0.918, and on USPS it also exceeds

both dense networks. It performs worse on letter at 0.783 against 0.830, which the fixed arity results explain, since letter has 26 classes and benefits from deeper or wider trees, and a depth-5 budget with an average path of 2.27 is too small to separate it. The dense networks remain the most accurate overall, which is expected based on the results obtained on the authors of the FFF networks.

The latency behaviour follows Section 4.1. MR-FFF evaluates only about one to two nodes per input against the ten routing steps of FFF, yet it runs 1.7 to 2.2 times slower, as Figure 6 shows. Each MR-FFF step does more control work than an FFF step, since it evaluates a K-way router and a halting score and gathers from a heap, and the models here are too small for this fixed cost to be hidden by FLOPs. To further saving time, the MR-FFF would require a larger datasete or a fused inference kernel.

Table 1: Ablation results. The reference is a depth-6 binary FFF. $\bar{L}$ is the average path length out of $D = 3$ and $f_{\max}$ the fraction of inputs reaching the deepest level.

Dataset	Config	Acc	Time (μs)	$\bar{L}$	$f_{\max}$
spambase	ref FFF D=6	0.8862	0.92	6.00	–
	fixed K=4, D=3	0.9401	1.11	3.00	–
	learned λ=0	0.9427	1.51	2.12	0.41
	learned λ=0.02	0.9427	1.49	1.03	0.02
USPS	ref FFF D=6	0.9325	3.00	6.00	–
	fixed K=4, D=3	0.9583	4.57	3.00	–
	learned λ=0	0.9544	5.51	1.69	0.36
	learned λ=0.02	0.9557	5.71	0.82	0.06
letter	ref FFF D=6	0.8246	0.52	6.00	–
	fixed K=8, D=2	0.9264	0.70	2.00	–
	learned λ=0	0.9258	0.86	2.27	0.69
	learned λ=0.1	0.8864	0.87	1.05	0.04
satimage	ref FFF D=6	0.8470	0.70	6.00	–
	fixed K=8, D=2	0.8825	0.77	2.00	–
	learned λ=0	0.8806	1.12	2.17	0.56
	learned λ=0.02	0.8762	1.13	0.71	0.02

Table 2: Benchmark results. FFF has depth 10, the dense networks have width 1024 with one or two hidden layers, and MR-FFF uses $K = 4$ and $D = 5$. The ratio is MR-FFF time / FFF time and $\bar{L}$ is out of 5.

	Accuracy				MR-FFF	
Dataset	FFF	MLP1	MLP2	MR-FFF	ratio	$\bar{L}$
spambase	0.8671	0.9496	**0.9513**	0.9227	1.69	1.14
USPS	0.9178	0.9570	0.9505	**0.9630**	2.18	1.76
letter	0.8300	0.9546	**0.9570**	0.7830	1.73	2.27
satimage	0.7767	0.8924	**0.9017**	0.8396	1.71	1.08

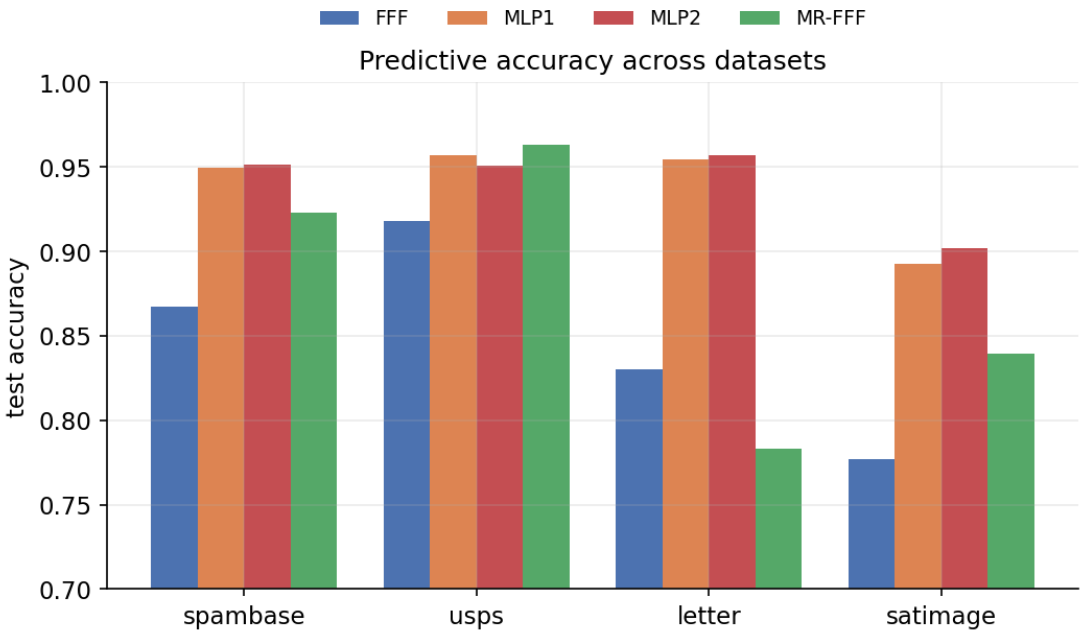

Figure 5: Benchmark accuracy. MR-FFF reaches or exceeds FFF on three datasets.

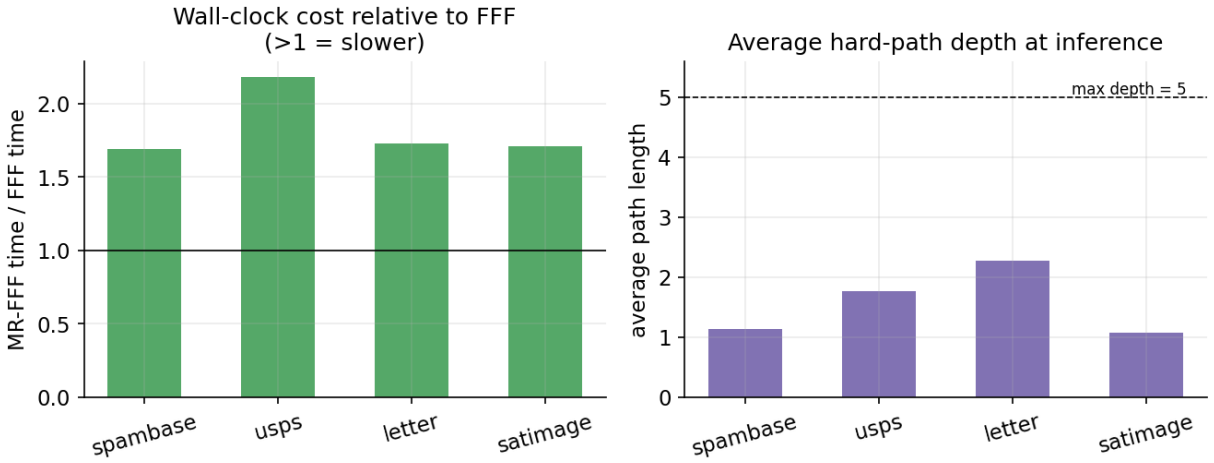

Figure 6: Left: MR-FFF inference time divided by FFF time. Right: average length of MR-FFF paths

5 Conclusion and Limitations

While MR-FFF performs better in terms of accuracy with respect to FFF networks, the 2 hidden layer MLP performs better in terms of average accuracy. Experiments show that MR-FFF has the best tradeoff in terms of accuracy and inference time. Optimization on the routing kernel are necessary to improve the time on small scale datasets, in order to make MR-FFF competitive to dense networks in every setting.

References

[1] P. Belcak and R. Wattenhofer. Fast Feedforward Networks. arXiv:2308.14711, 2023.